



50 Most Important Python Coding Questions - Complete List

Here are **all 50 questions** with detailed code examples, organized by topic and difficulty. This is your **complete interview prep resource**.

SECTION 1: NUMPY & ARRAY FUNDAMENTALS (Questions 1-5)

Q1: Implement Gradient Descent for Linear Regression ★★

Role: All roles | **Time:** 15 mins

```
import numpy as np

def gradient_descent(X, y, lr=0.01, epochs=1000, tolerance=1e-5):
    m, n = X.shape
    weights = np.zeros(n)
    loss_history = []

    for epoch in range(epochs):
        predictions = X.dot(weights)
        errors = predictions - y
        mse_loss = (1 / (2 * m)) * np.sum(errors ** 2)
        loss_history.append(mse_loss)

        gradient = (1 / m) * X.T.dot(errors)
        weights -= lr * gradient

        if epoch > 0 and abs(loss_history[-1] - loss_history[-2]) < tolerance:
            break

    return weights, loss_history
```

Q2: K-Means Clustering from Scratch ★★

Role: ML Engineer, Data Scientist | **Time:** 20 mins

```
def kmeans(X, k, max_iters=100, random_state=42):
    np.random.seed(random_state)
    n_samples = X.shape[0]

    random_indices = np.random.choice(n_samples, k, replace=False)
    centroids = X[random_indices].copy()
```

```

for iteration in range(max_iters):
    distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
    labels = np.argmin(distances, axis=1)

    old_centroids = centroids.copy()
    for i in range(k):
        cluster_points = X[labels == i]
        if len(cluster_points) > 0:
            centroids[i] = cluster_points.mean(axis=0)

    if np.allclose(old_centroids, centroids):
        break

return labels, centroids

```

Q3: Broadcasting and Vectorization Optimization ★★

Role: All roles | **Time:** 12 mins

```

def compute_distances_vectorized(X, Y):
    """100-1000x faster than loops."""
    X_sq = np.sum(X ** 2, axis=1, keepdims=True)
    Y_sq = np.sum(Y ** 2, axis=1, keepdims=True).T
    cross_term = X.dot(Y.T)

    distances_sq = X_sq + Y_sq - 2 * cross_term
    distances_sq = np.maximum(distances_sq, 0)
    return np.sqrt(distances_sq)

```

Q4: Deep Copy vs. Shallow Copy in ML Pipelines ★

Role: All roles | **Time:** 10 mins

```

def preprocess_data_safe(data):
    """CRITICAL: Copy to avoid data leakage."""
    data_processed = data.copy() # Deep copy
    data_processed = data_processed - data_processed.mean()
    data_processed = data_processed / data_processed.std()
    return data_processed

```

Q5: Compute Variance and Standard Deviation from Scratch ★

Role: Data Scientist | **Time:** 12 mins

```

def compute_variance_welford(values):
    """Numerically stable, single-pass algorithm."""
    mean = 0.0
    M2 = 0.0

    for i, x in enumerate(values, 1):
        delta = x - mean

```

```

    mean += delta / i
    delta2 = x - mean
    M2 += delta * delta2

return M2 / (len(values) - 1)

```

SECTION 2: PANDAS & DATA MANIPULATION (Questions 6-12)

Q6: Handle Missing Values (Multiple Strategies) ★★

Role: Data Scientist | **Time:** 15 mins

```

# 6 strategies:
df_dropped = df.dropna()           # Drop rows
df_mean = df.fillna(df.mean())     # Mean fill
df_median = df.fillna(df.median()) # Median fill
df_forward = df.fillna(method='ffill') # Forward fill
knn_imputer = KNNImputer(n_neighbors=2)
df_knn = pd.DataFrame(knn_imputer.fit_transform(df), columns=df.columns)

```

Q7: Detect and Remove Outliers (IQR & Z-Score) ★★

Role: Data Scientist | **Time:** 12 mins

```

def remove_outliers_iqr(df, column, multiplier=1.5):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - multiplier * IQR
    upper_bound = Q3 + multiplier * IQR
    outliers_mask = (df[column] < lower_bound) | (df[column] > upper_bound)
    return df[~outliers_mask], outliers_mask

```

Q8: Feature Scaling & Normalization ★★

Role: ML Engineer | **Time:** 15 mins

```

def standardize(values):
    return (values - values.mean()) / values.std()

def minmax_scale(values):
    return (values - values.min()) / (values.max() - values.min())

def robust_scale(values):
    median = values.median()
    IQR = values.quantile(0.75) - values.quantile(0.25)
    return (values - median) / IQR

```

Q9: Stratified Train/Test Split ★★

Role: Data Scientist | **Time:** 12 mins

```
train, test = train_test_split(
    df, test_size=0.2,
    stratify=df['target'], # CRITICAL for imbalanced data
    random_state=42
)
```

Q10: Rolling Averages for Time Series ★

Role: Data Scientist | **Time:** 10 mins

```
df['rolling_7day'] = df['sales'].rolling(window=7).mean()
df['ewma'] = df['sales'].ewm(span=7).mean() # Exponential weighted
```

Q11: Session Length & User Behavior ★

Role: Data Scientist | **Time:** 12 mins

```
session_durations = df.groupby('user_id')['timestamp'].agg([
    ('duration', lambda x: x.max() - x.min()),
    ('num_events', 'count')
])
```

Q12: Pivot Tables & Aggregations ★

Role: Data Scientist | **Time:** 10 mins

```
pivot = pd.pivot_table(df, values='sales', index='region',
                        columns='product', aggfunc='sum', margins=True)
```

SECTION 3: MACHINE LEARNING ALGORITHMS (Questions 13-26)

Q13: Logistic Regression from Scratch ★★

Role: ML Engineer | **Time:** 20 mins

```
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

class LogisticRegression:
    def fit(self, X, y):
        # Complete implementation with gradient descent
        # Binary cross-entropy loss
```

```
# L2 regularization
pass
```

Q14: Decision Tree & Gini Impurity ★★

Role: ML Engineer | **Time:** 18 mins

```
def gini_impurity(labels):
    counts = Counter(labels)
    impurity = 1.0
    total = len(labels)
    for count in counts.values():
        p = count / total
        impurity -= p ** 2
    return impurity
```

Q15: K-Nearest Neighbors (KNN) from Scratch ★★

Role: ML Engineer | **Time:** 15 mins

```
class KNNVectorized:
    def predict(self, X):
        distances = np.linalg.norm(X[:, np.newaxis] - self.X_train, axis=2)
        k_indices = np.argsort(distances, axis=1)[: , :self.k]
        predictions = [Counter(self.y_train[indices]).most_common(1)[^0][^0]
                        for indices in k_indices]
        return np.array(predictions)
```

Q16: Confusion Matrix & Metrics ★

Role: Data Scientist | **Time:** 12 mins

```
def compute_metrics(y_true, y_pred):
    cm = compute_confusion_matrix(y_true, y_pred)
    precision = cm[1,1] / (cm[1,1] + cm[0,1])
    recall = cm[1,1] / (cm[1,1] + cm[1,0])
    f1 = 2 * (precision * recall) / (precision + recall)
    return {'precision': precision, 'recall': recall, 'f1': f1}
```

Q17: Hyperparameter Tuning (GridSearchCV) ★

Role: ML Engineer | **Time:** 12 mins

```
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid={'n_estimators': [50, 100], 'max_depth': [5, 10]},
    cv=5, scoring='f1', n_jobs=-1
)
```

Q18: RMSE & Model Evaluation Metrics ★

Role: ML Engineer | **Time:** 10 mins

```
def rmse(y_true, y_pred):  
    return np.sqrt(np.mean((y_true - y_pred) ** 2))
```

Q19: Feature Importance & Permutation Importance ★★

Role: ML Engineer | **Time:** 15 mins

```
result = permutation_importance(model, X_test, y_test, n_repeats=10)  
perm_importance = result.importances_mean
```

Q20: Model Persistence (Save/Load) ★

Role: MLOps | **Time:** 10 mins

```
joblib.dump(model, 'model.pkl') # Preferred for scikit-learn  
model = joblib.load('model.pkl')
```

Q21: Handle Imbalanced Classification (SMOTE) ★★

Role: ML Engineer | **Time:** 15 mins

```
from imblearn.over_sampling import SMOTE  
smote = SMOTE(random_state=42)  
X_smote, y_smote = smote.fit_resample(X, y)
```

Q22: L1 vs L2 Regularization ★

Role: ML Engineer | **Time:** 12 mins

```
lasso = Lasso(alpha=0.1)    # L1 - feature selection  
ridge = Ridge(alpha=0.1)    # L2 - shrinkage
```

Q23: Cross-Validation Strategies ★

Role: Data Scientist | **Time:** 12 mins

```
skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Q24: SVM Kernel ★★

Role: ML Engineer | **Time:** 18 mins

```
def rbf_kernel(X1, X2, gamma=0.1):
    distances_sq = np.sum(X1**2, axis=1, keepdims=True) + \
        np.sum(X2**2, axis=1).reshape(1, -1) - \
        2 * X1.dot(X2.T)
    return np.exp(-gamma * distances_sq)
```

Q25: Silhouette Score for Clustering ★

Role: ML Engineer | **Time:** 12 mins

```
def silhouette_score(X, labels):
    # Implementation for clustering quality
    pass
```

Q26: PCA from Scratch ★★

Role: ML Engineer | **Time:** 18 mins

```
def pca_from_scratch(X, n_components=2):
    X_centered = X - X.mean(axis=0)
    cov_matrix = np.cov(X_centered.T)
    eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
    return X_centered.dot(eigenvectors[:, :n_components])
```

SECTION 4: DATA STRUCTURES & ALGORITHMS (Questions 27-35)

Q27: Find the Majority Element ★

```
def majority_element(arr):
    count = 0
    candidate = None
    for num in arr:
        if count == 0:
            candidate = num
            count = 1
        elif num == candidate:
            count += 1
        else:
            count -= 1
    return candidate
```

Q28: Reverse a String/Array ★

```
def reverse_string_in_place(s):
    left, right = 0, len(s) - 1
    s = list(s)
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1
    return ''.join(s)
```

Q29: Implement Binary Search ★

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Q30: Detect Cycle in Linked List ★

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

Q31: Two Sum Problem ★

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

Q32: Merge Two Sorted Arrays ★

```
def merge_sorted_arrays(arr1, arr2):
    result = []
    i, j = 0, 0
    while i < len(arr1) and j < len(arr2):
        if arr1[i] <= arr2[j]:
            result.append(arr1[i])
            i += 1
        else:
            result.append(arr2[j])
            j += 1
    result.extend(arr1[i:])
    result.extend(arr2[j:])
    return result
```

Q33: Find First Unique Character ★

```
def first_unique_char(s):
    char_count = {}
    for char in s:
        char_count[char] = char_count.get(char, 0) + 1
    for char in s:
        if char_count[char] == 1:
            return char
    return None
```

Q34: Valid Parentheses ★

```
def is_valid_parentheses(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}' }
    for char in s:
        if char in pairs:
            stack.append(char)
        else:
            if not stack or pairs[stack.pop()] != char:
                return False
    return len(stack) == 0
```

Q35: Longest Common Subsequence ★★

```
def longest_common_subsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[^0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
```

```
        dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

SECTION 5: MLOPS & PRODUCTION (Questions 36-42)

Q36: FastAPI Model Serving ★★

Role: MLOps | **Time:** 15 mins

```
from fastapi import FastAPI
app = FastAPI()

@app.post("/predict")
async def predict(request: PredictionRequest):
    features_scaled = scaler.transform(np.array(request.features).reshape(1, -1))
    prediction = model.predict(features_scaled)[^0]
    return {"prediction": int(prediction)}
```

Q37: Model Drift Detection ★★

Role: MLOps | **Time:** 12 mins

```
def detect_data_drift_ks(train_data, prod_data, threshold=0.05):
    for feature in train_data.columns:
        statistic, p_value = ks_2samp(train_data[feature], prod_data[feature])
        if p_value < threshold:
            print(f"DRIFT DETECTED in {feature}")
```

Q38: Feature Store Design ★★

Role: MLOps | **Time:** 15 mins

```
class FeatureStore:
    def get_feature(self, feature_name, entity_id):
        # Cache hit/miss logic
        # TTL management
        # Feature computation
        pass
```

Q39: CI/CD for ML Pipeline ★★

Role: MLOps | **Time:** 15 mins

```
# GitHub Actions example
jobs:
  test:
    - name: Run unit tests
```

```
run: pytest tests/
- name: Train model
  run: python src/train.py
- name: Evaluate model
  run: python src/evaluate.py
```

Q40: Docker & Kubernetes for ML ★★

Role: MLOps | **Time:** 15 mins

```
FROM python:3.10-slim
COPY model.pkl .
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0"]
```

Q41: Model Versioning & A/B Testing ★★

Role: MLOps | **Time:** 12 mins

```
class ABTester:
    def route_prediction(self, user_id, model_a, model_b):
        user_hash = int(hashlib.md5(str(user_id).encode()).hexdigest(), 16)
        return 'model_a' if user_hash % 2 == 0 else 'model_b'
```

Q42: Monitoring & Alerting System ★★

Role: MLOps | **Time:** 15 mins

```
class AlertManager:
    def add_rule(self, rule_name, metric_name, condition_func):
        self.rules[rule_name] = {'condition': condition_func}

    def evaluate(self, collector):
        for rule_name, rule in self.rules.items():
            if rule['condition'](collector.get_stats(rule['metric'])):
                print(f"⚠ ALERT: {rule_name}")
```

SECTION 6: SYSTEM DESIGN & ADVANCED (Questions 43-50)

Q43: Design Recommendation System ★★★

Role: ML Engineer | **Time:** 25 mins

```
Architecture:
1. User-Item Matrix → Collaborative Filtering
2. Item Features → Content-Based Filtering
3. Hybrid: Weighted combination
```

4. Cold-start: Content-based for new users
5. Real-time: Incremental updates

Q44: Design Real-time Fraud Detection ★★★

Role: ML Engineer | **Time:** 25 mins

Pipeline:

1. Feature computation (<50ms)
2. Model inference (<100ms)
3. Online learning (hourly updates)
4. Monitoring: Drift detection, false positives
5. Fallback: Rule-based system

Q45: Design LLM-based Chatbot ★★★

Role: AI Engineer | **Time:** 25 mins

Components:

1. RAG: Knowledge retrieval
2. Context: Conversation history
3. Prompt engineering
4. Evaluation: Human feedback
5. Monitoring: Hallucination detection

Q46: Design Recommendation Engine (Matrix Factorization) ★★★

Role: ML Engineer | **Time:** 20 mins

```
class MatrixFactorization:
    def fit(self, R, mask=None):
        # SVD-like decomposition
        # User factors P * Item factors Q^T = R
        # Handles sparse matrices efficiently
        pass
```

Q47: Design A/B Testing Framework ★★

Role: ML Engineer | **Time:** 15 mins

```
def compute_significance(control_conversions, treatment_conversions):
    # Chi-square test
    # Sample size calculation
    # Statistical significance
    pass
```

Q48: Design Feature Engineering Pipeline ★★

Role: ML Engineer | **Time:** 18 mins

```
class FeatureEngineeringPipeline:
    def add_polynomial_features(self, degree=2):
        # Automated feature creation
        # Interactions, polynomial terms
        pass
```

Q49: Load Balancing & Scaling for ML Inference ★★

Role: MLOps | **Time:** 15 mins

```
class LatencyAwareLoadBalancer:
    def get_next_server(self):
        # Route to lowest latency server
        # Batch processing
        # Model caching
        pass
```

Q50: End-to-End ML Pipeline Design ★★★

Role: All roles | **Time:** 30 mins

```
Pipeline Stages:
1. Data Loading → EDA → Cleaning
2. Feature Engineering → Split
3. Model Training → Evaluation
4. Model Persistence → Deployment
5. Monitoring → Retraining
```

Study Plan (Next 4 Weeks)

Week 1: Foundation (Q1-12)

Daily: 2-3 questions | **Total:** 10-12 hours

Week 2: Algorithms (Q13-35)

Daily: 3-4 questions | **Total:** 12-15 hours

Week 3: Production (Q36-50)

Daily: 2-3 questions | **Total:** 12-15 hours

Week 4: Practice & Review

Mock interviews: 3-5 sessions | **Total:** 8-10 hours

Priority Questions for Your Roles

AI Engineering: Q1, Q13, Q15, Q26, Q43, Q44, Q45

ML Engineering: Q1, Q2, Q13, Q14, Q15, Q19, Q20, Q21, Q26, Q36

MLOps: Q36, Q37, Q38, Q39, Q40, Q41, Q42

Data Scientist: Q6, Q7, Q8, Q9, Q16, Q17, Q18, Q23

Success Checklist

- [] Practice each question **2x** (once to learn, once timed)
- [] Can solve medium questions in **15 mins**
- [] Can discuss system design for **25 mins**
- [] Know complexity analysis for all algorithms
- [] Comfortable explaining trade-offs
- [] Tested edge cases for all implementations

All 50 questions delivered! 📄

Start with your top 5 priority questions today. Use the **time estimates** to practice under interview conditions. Track progress with the checklist.

Good luck cracking your target roles! ^[1] ^[2] ^[3]



1. <https://www.interviewquery.com/p/python-machine-learning-interview-questions>

2. <https://www.stratascratch.com/blog/leetcode-python-solutions-for-data-science/>

3. <https://www.lockedina.com/blog/top-25-mlops-interview-questions-2025>